

Al-Aqsa University

Faculty of Computers and IT

Engineering and Networks Dept.



جامعة الأقصى
كلية الحاسبات وتكنولوجيا المعلومات
قسم الهندسة والشبكات

Operating Systems for Intelligent Systems (ENGS2306)

Lecture 5: Process Management in Linux

Lecturer: Eng. Mohammed E. Alhams

Semester: Second Semester 2024/2025

Lecture 5: Process Management in Linux

Today's Objectives:

1. Understand what processes are and their different states.
 2. Learn to view and monitor running processes using ps, top, and htop.
 3. Learn how to control processes using kill, nice, and renice.
 4. Apply these skills to scenarios in intelligent systems (IoT/Android).
-

Recap: From Users to Processes

- In **Lecture 4**, we learned to create and manage user and group accounts. We established *who* can access the system.
 - **Today's Connection:** When a user runs a program, it becomes a **process**.
 - Every single process on the system is **owned by a user**.
 - The process inherits the permissions of the user who started it.
 - You can manage your own processes, but you need sudo to manage processes owned by other users or the system itself.
-

What is a Process?

A **process** is an instance of a running program. If a program is a blueprint, a process is the factory actively building something from that blueprint.

Key Concepts:

- **Process ID (PID):** A unique number the kernel assigns to each process. This is how you identify and control a specific process.
- **Parent Process ID (PPID):** The PID of the process that created this one. All processes form a tree structure back to the first process, init (PID 1).

For Intelligent Systems: On a Raspberry Pi running a smart camera application, the script reading the camera feed is one process, while the web server displaying the video stream is another. They are separate, manageable tasks.

Understanding Process States

A process is not always using the CPU. It moves between several states:

- **R (Running or Runnable):** The process is either currently using the CPU or is in the queue, ready to run.
 - **S (Interruptible Sleep):** The process is waiting for an event (e.g., user input, network data). This is the most common state for processes.
 - **D (Uninterruptible Sleep):** The process is in a deep sleep, usually waiting for I/O (like reading from a disk) and cannot be interrupted by signals.
 - **Z (Zombie):** A defunct process. It has finished executing, but its entry remains in the process table until its parent process "collects" its exit status. Zombies don't use CPU, but many can signal a problem with a parent application.
 - **T (Stopped):** The process has been paused by a user or a signal.
-

Viewing Processes with ps

The ps (process status) command gives you a snapshot of current processes.

The Most Useful Combination: ps aux

- a = Show processes for **all** users.
- u = Display in a **user-oriented** format.
- x = Show processes not attached to a terminal (background services).

Example Output:

```
$ ps aux
USER          PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root           1  0.0  0.1 169608 12824 ?        Ss   10:30   0:02 /sbin/init
mo-alhams    1234  0.2  0.5 876543 45123 ?        Sl   10:32   0:15 /usr/bin/gnome-shell
mo-alhams    5678  0.0  0.0  12345   890 pts/0    R+   11:05   0:00 ps aux
```

- **Key Columns:** USER, PID, %CPU, %MEM, STAT, and COMMAND.

Interactive Monitoring with top

top provides a real-time, dynamic dashboard of your system's processes, sorted by CPU usage by default.

How to Run: Simply type top in your terminal.

Key Features:

- Live-updating list of processes.
- Summary of system resources (CPU, Memory, Load Average).

Interactive Commands (while top is running):

- Press k to **kill** a process (it will ask for the PID).
 - Press r to **renice** a process (change its priority).
 - Press q to **quit**.
-

User-Friendly Monitoring with htop

htop is an improved, color-coded, and more interactive version of top.

Advantages:

- Visual bars for CPU, memory, and swap usage.
- Scroll vertically and horizontally through the process list with arrow keys.
- Use function keys for common actions (e.g., F9 to Kill, F7/F8 to Renice).

Installation (not installed by default):

```
# First, update your package list  
sudo apt update
```

```
# Then, install htop  
sudo apt install htop
```

Once installed, simply run htop.

Controlling Processes: kill and Signals

The kill command does not just "kill"; it sends a **signal** to a process. The process can choose how to react.

Syntax: kill [signal_option] <PID>

Most Important Signals:

- **SIGTERM (15):** The default signal. A polite request asking the process to **terminate** gracefully (save files, clean up, then exit).
kill 1234
- **SIGKILL (9):** The "force quit" signal. It tells the kernel to terminate the process immediately, without giving it a chance to clean up. Use this as a last resort for frozen applications.
kill -9 1234 or kill -SIGKILL 1234

Remember: To kill a process owned by another user or the system, you must use sudo.

Managing Priority: nice and renice

On a resource-limited intelligent system (like an IoT device), you must ensure critical tasks get priority.

- **Niceness:** A value from -20 (highest priority) to +19 (lowest priority). The default is 0. A higher value is "nicer" to other processes (lower priority).

Commands:

1. **nice:** Start a **new** process with a specific priority.

```
# Start a data-logging script with low priority
nice -n 15 ./log_sensor_data.sh
```

2. **renice:** Change the priority of a **running** process.

```
# Make a running process (PID 5678) a higher priority
# Note: Only root can increase priority (make nice value lower)
sudo renice -5 -p 5678
```

Practical Scenario: Managing a Task

Follow these steps to practice the full lifecycle of process management.

1. Open your terminal.
2. Start a simple, long-running process in the background using &.
sleep 600 &
3. The terminal will print its PID. Let's assume it is [1] 8899.

4. Find your sleep process using ps.
ps aux | grep sleep
 5. Open htop and find the sleep process. Note its PID and NI (nice) value.
 6. Let's make it lower priority (nicer). Replace 8899 with your actual PID.
renice 10 -p 8899
 7. Check htop again. The NI value for the process should now be 10.
 8. Now, terminate it politely with SIGTERM.
kill 8899
 9. You should see a 'Terminated' message in your terminal.
 10. Verify it is gone with ps aux | grep sleep. The command should return nothing.
-

The Idea Behind the Assignment

Why is this critical for Intelligent Systems?

- **IoT Performance & Stability:** An IoT gateway runs multiple services. If a non-critical data-logging process consumes 100% of the CPU, it could prevent a critical security alert from being sent. Using nice on the logging process ensures the alert gets priority. If the logging process freezes, kill allows you to stop it remotely without rebooting the entire device.
 - **Debugging Embedded Systems:** When an application on an embedded device or Android phone becomes unresponsive, tools like top and htop (often accessed via a debug connection like adb shell) are the first step to identify the resource-hogging process.
 - **Resource Management:** By understanding process states and priorities, you can design more robust and reliable intelligent systems that manage their own resources effectively, even on low-power hardware.
-

Assignment 4: Process Exploration & Management (12%)

Objective: To practice monitoring, prioritizing, and terminating processes on your system.

Tasks:

1. **Explore:** Run htop. Identify 3-4 key system processes (e.g., systemd, gnome-shell, sshd). Take a screenshot of your htop window.
2. **Create & Identify:** Start a process that sleeps for 5 minutes (300 seconds) in the background: sleep 300 &. Use ps -ef | grep sleep to find its PID.

3. **Prioritize:** Use the renice command to change its priority to 12.
 4. **Verify Priority:** Use the command `ps -o pid,ni,comm -p <YOUR_PID>` to clearly show the new niceness value. Include this command and its output in your report.
 5. **Terminate:** Use the kill command to terminate the sleep process.
 6. **Verify Termination:** Run `ps -ef | grep sleep` again to confirm the process is gone.
 7. **Report:** Write a 1-2 paragraph report on **one** of the following concepts:
 - The difference between the SIGTERM and SIGKILL signals, and a practical scenario for using each one.
 - What a 'zombie' process is, why it occurs, and how the system eventually cleans it up.
-

Assignment Submission Instructions

- **File Name:** Name your submission file exactly as follows:
studentnum-studentname-Assignment4.pdf
 - *Example:* 13012070xx-MohammedEmadAlhams-Assignment4.pdf
 - **Terminal Prompt:** As with the previous assignment, your name must appear in the terminal prompt for all screenshots and command outputs.
 - **File Format:** Your submission must be a **single PDF file** containing:
 - The htop screenshot.
 - The commands you used for each step, with their outputs.
 - A brief description of each command's function.
 - Your final report paragraph.
 - **Academic Integrity:**
 - **Warning:** Any two similar submissions will be considered an attempt to cheat and will result in a grade of **ZERO**.
 - **Warning:** Any AI-generated solutions or reports will also result in a grade of **ZERO**.
-

Next Steps & Resources

Practice This Week:

- Keep htop running in a terminal window while you work. Watch how processes are created and terminated as you open and close applications.
- Be careful with kill and sudo! Only experiment on processes you started yourself, like sleep.

Next Week: Lecture 6 - Bash Scripting Basics

- We will learn to automate the commands from the past weeks. Imagine a script that automatically finds and lowers the priority of any process consuming too much CPU!

Gaza Connectivity Tips:

- Download this lecture PDF and any related videos now.
- Complete your assignment fully offline. You only need to connect to submit the final PDF.

Resources:

- Linux Process Management Tutorial: <https://www.linode.com/docs/guides/linux-process-management/>
- Read the manual pages: man ps, man top, man kill
- Course Materials on Moodle